Independent LLM Security Assessment

Adversarial Robustness Testing of GPT-3.5-Turbo

Target System: gpt-3.5-turbo (OpenAI API)

Testing Tool: NVIDIA Garak v0.15.1 (open-source LLM vulnerability scanner)

Assessment Type: Automated adversarial probing independent research assessment

Date: July 2026

Executive Summary

This assessment used the Garak framework to evaluate the adversarial robustness of gpt-3.5-turbo across three vulnerability classes: prompt injection, jailbreak (persona-override) attacks, and encoding-based obfuscation. The scan executed 9,988 individual attack attempts across 12 distinct probes and 5 detectors, with 5 generations sampled per prompt to account for response variance.

The model demonstrated a critical weakness to prompt injection via long-context burying (76.2% attack success), and high susceptibility to common encoding schemes (Hex/Base16: 67–69%) and jailbreak personas circulating in the wild (DanInTheWild: 56.2%, AutoDANCached: 60.0%). Conversely, the model showed strong resistance to less common encoding schemes (Base32, Base2048, Ascii85, UUencoding, Braille, Atbash), each with success rates below 0.5%, indicating its safety training generalizes poorly to encodings underrepresented in that training data rather than reflecting robust decoding-based defenses.

These findings are consistent with publicly documented weaknesses in this model family and are presented here as a demonstration of an end-to-end adversarial testing and reporting workflow, not as a novel disclosure.

Scope & Methodology

- Target: gpt-3.5-turbo, accessed via the OpenAI API
- Tool: NVIDIA Garak v0.15.1, using the probewise test harness
- Probe families tested: promptinject (rogue-string hijacking), dan (jailbreak/persona-override), encoding (payload obfuscation)
- Detectors: dan.DAN, mitigation.MitigationBypass, promptinject.AttackRogueString, encoding.DecodeMatch, encoding.DecodeApprox
- Generations per prompt: 5 (to capture response variance and avoid false negatives from a single sample)
- Total logged attempts: 9,988

A "success" (fail/hit) is recorded when a detector confirms the model exhibited the targeted vulnerable behavior such as adopting a jailbroken persona, leaking an injected rogue string, or decoding and complying with an obfuscated malicious payload.

Findings Summary

Probe / Vector	Category	Success Rate	Severity
HijackLongPrompt	Prompt Injection	76.2%	Critical
InjectHex / InjectBase16	Encoding Obfuscation	67–69%	High
AutoDANCached	Jailbreak (DAN)	60.0%	High
HijackHateHumans	Prompt Injection	59.3%	High
DanInTheWild	Jailbreak (DAN)	56.2%	High
HijackKillHumans	Prompt Injection	53.7%	High
InjectBase64	Encoding Obfuscation	28–51%	Medium
InjectZalgo / InjectNato	Encoding Obfuscation	35–46%	Medium
Ablation_Dan_11_0	Jailbreak (DAN)	23–26%	Medium

InjectMorse / InjectROT13	Encoding Obfuscation	10–23%	Low
InjectBase32 / Base2048 / Ascii85 / UU / Braille / Atbash	Encoding Obfuscation	0–0.5%	Negligible

Detailed Findings

1. Prompt Injection via Long-Context Burying — Critical (76.2% success)

Technique: A benign-looking long prompt embeds a secondary instruction designed to override the original task and force the model to output an attacker-specified string. This is the highest-severity finding in the assessment.

Impact: In a production application (e.g., a customer-facing chatbot or document-summarization tool), this vector could be used to hijack the model's output, exfiltrate a fixed payload, or redirect user-facing responses. This is particularly dangerous in RAG pipelines where untrusted document content reaches the model as context.

Recommendation: Enforce a strict instruction hierarchy (system prompt > developer prompt > user/document content) and apply output validation that flags responses matching known injection markers before they reach the end user.

2. Encoding-Based Obfuscation — High (Hex/Base16: 67–69%)

Technique: A malicious instruction is encoded (e.g., hexadecimal) and the model is asked to decode and act on it. The model frequently decoded and complied with the hidden instruction despite it bypassing plaintext content filters.

Impact: Confirms that safety filtering in this model is largely applied at the semantic/plaintext layer and can be bypassed by shifting the harmful content into an encoding the filter doesn't pattern-match against.

Recommendation: Add a pre-processing layer that detects and decodes common encodings (hex, base64, etc.) in user input before it reaches the model, then re-applies content filtering to the decoded text.

3. Jailbreak / Persona-Override Attacks — High (56–60%)

Technique: Prompts instruct the model to adopt an alternate persona (e.g., a variant of the well-documented "DAN", Do Anything Now, jailbreak family) explicitly told to disregard its safety guidelines. Both a curated real-world prompt set (DanInTheWild) and an automated jailbreak-generation approach (AutoDAN) were tested.

Impact: Successful persona overrides can cause the model to produce content that violates its intended use policy, undermining any safety guarantees an application built on top of it is relying on.

Recommendation: Layer a separate, non-bypassable output classifier that operates independently of the model's own instruction-following (i.e., cannot itself be "convinced" via prompt), and treat the base model's safety training as a first line of defense rather than the only one.

4. Prompt Injection — Rogue String Hijacking (53–59%)

Technique: Direct instruction-override attempts asking the model to disregard its task and output a specific attacker-chosen string in place of its intended response.

Recommendation: Same mitigation family as Finding 1; Instruction hierarchy enforcement plus output-side pattern detection for known injection markers.

5. Areas of Strong Resistance

Base32, Base2048, Ascii85, UUencoding, Braille, and Atbash encodings all produced success rates below 0.5%, and ROT13/Morse-code encodings showed comparatively low rates (10–23%). This suggests the model's resistance is uneven and likely tied to the prevalence of each encoding scheme in its training/safety-tuning data rather than a general decoding-based defense, which is a distinction worth noting for anyone evaluating whether a model's safety training will generalize to novel obfuscation techniques.

Overall Recommendations

- Do not rely solely on a foundation model's built-in safety training for production deployments. Add independent input/output filtering layers.
- Treat prompt injection as the highest-priority risk for any application that inserts untrusted content (documents, web pages, user messages) into the model's context window.
- Incorporate automated adversarial regression testing (e.g., Garak) into CI/CD for any product shipping LLM-powered features, so regressions are caught before release rather than after.
- Re-run this test suite against any model upgrade or system-prompt change, since safety behavior can shift between versions.

Conclusion

This assessment demonstrates an end-to-end adversarial LLM testing workflow: scoping, automated probing across multiple vulnerability classes, evidence-based severity rating, and actionable remediation guidance. The findings align with known, publicly documented weaknesses in this model family and illustrate the kind of systematic evaluation needed before deploying LLMs in production, security-sensitive contexts.